

Sprint 8 Notes: Buttons, Validation, and Form States

Sprint 8 Goal

The goal of Sprint 8 is to improve basic HTML forms.

In Sprint 7, you learned how to create a basic form using:

- `form`
- `action`
- `input`
- `label`
- `id`
- `name`
- `placeholder`

But a basic form is not complete yet.

A useful form usually needs:

- a button to submit the form
- validation to check simple mistakes
- different input states, such as disabled or readonly

Sprint 8 teaches these important form features.

By the end of Sprint 8, you should understand:

- what buttons do inside forms
 - the difference between `button` and input buttons
 - the three main button types
 - what native browser validation is
 - how `required` works
 - how `type="email"` checks email format
 - how `minlength` and `maxlength` control input length
 - what default, focused, disabled, and readonly states mean
 - the difference between `disabled` and `readonly`
 - how to build a validated beginner form
 - how to avoid common Sprint 8 mistakes
-

1. Where Sprint 8 Fits in Your HTML Progress

Before Sprint 8, you learned how to display and organize web page content.

Sprint 1

You learned HTML basics:

```
<h1>My Page</h1>
<p>This is a paragraph.</p>

```

You learned:

- elements
- tags
- attributes
- void elements
- `id`
- `class`
- basic image structure

Sprint 2

You learned how to write a complete HTML document:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>My Page</title>
</head>
<body>
  <h1>My Page</h1>
</body>
</html>
```

You learned:

- `DOCTYPE`
- `html`
- `head`

- `body`
- metadata
- page title
- visible page content

Sprint 3

You learned how to mark text based on meaning:

```
<p><strong>Warning:</strong> Do not share your password.</p>
<p>The term <i>semantic HTML</i> means HTML with meaning.</p>
```

Sprint 4

You learned links and navigation:

```
<nav>
  <a href="index.html">Home</a>
  <a href="about.html">About</a>
  <a href="contact.html">Contact</a>
</nav>
```

Sprint 5

You learned media:

```

```

Sprint 6

You learned semantic page structure:

```
<header>
  <h1>Code Learning Hub</h1>
</header>

<main>
  <section>
    <h2>Featured Courses</h2>
    <p>Learn HTML step by step.</p>
```

```
</section>
</main>
```

Sprint 7

You learned basic forms:

```
<form action="">
  <label for="email">Email Address:</label>
  <input type="email" id="email" name="email">
</form>
```

Sprint 8

Now you are improving forms.

Simple idea:

Sprint 7 taught you how to create form fields. Sprint 8 teaches you how to submit, validate, and control those fields.

2. What Sprint 8 Adds to Forms

A Sprint 7 form can collect input, but it is still very basic.

Example Sprint 7 form:

```
<form action="">
  <label for="email">Email Address:</label>
  <input type="email" id="email" name="email">
</form>
```

This form has a field, but it has no button.

The user can type an email, but there is no clear way to submit the form.

Sprint 8 adds:

```
<button type="submit">Submit Form</button>
```

Sprint 8 also adds validation:

```
<input type="email" id="email" name="email" required>
```

Now the browser can check:

- Is the field empty?
- Does the value look like an email address?

Sprint 8 also adds form states:

```
<input type="text" value="Website" readonly>  
<input type="text" disabled>
```

These states control how the user can interact with the field.

3. Buttons in Forms

A button lets the user perform an action.

In forms, the most common action is submitting the form.

Basic submit button:

```
<button type="submit">Submit form</button>
```

Simple meaning:

```
type="submit"
```

means:

When the user clicks this button, submit the form.

Example:

```
<form action="">  
  <label for="email">Email Address:</label>  
  <input type="email" id="email" name="email">
```

```
<button type="submit">Submit form</button>
</form>
```

This form has:

- a form container
- a label
- an email input
- a submit button

The button gives the user a clear way to send the form.

4. The `button` Element

The `button` element creates a clickable button.

Basic pattern:

```
<button type="submit">Register</button>
```

The text between the opening and closing tags is visible on the button.

In this example, the user sees:

Register

The full button is:

```
<button type="submit">Register</button>
```

Parts:

Part	Meaning
<code><button></code>	starts the button element
<code>type="submit"</code>	tells the browser what kind of button this is
<code>Register</code>	visible button text
<code></button></code>	closes the button element

Simple rule:

Use `button` when you want a flexible button that can contain text or other content.

5. Button Type Values

The `type` attribute tells the browser what the button should do.

There are three important button types:

- `submit`
 - `reset`
 - `button`
-

6. `type="submit"`

A submit button sends the form data.

Example:

```
<button type="submit">Register</button>
```

Simple meaning:

Submit this form.

Full example:

```
<form action="/register">
  <label for="full-name">Full Name:</label>
  <input type="text" id="full-name" name="full-name">

  <button type="submit">Register</button>
</form>
```

When the user clicks **Register**, the browser tries to submit the form to:

```
/register
```

Beginner rule:

Use `type="submit"` when the button should send the form.

7. `type="reset"`

A reset button clears the form back to its starting values.

Example:

```
<button type="reset">Reset form</button>
```

Simple meaning:

Clear the form values and return them to the original values.

Example:

```
<form action="">
  <label for="full-name">Full Name:</label>
  <input type="text" id="full-name" name="full-name">

  <button type="reset">Reset form</button>
</form>
```

If the user types their name and clicks the reset button, the browser clears the typed value.

Be Careful with Reset Buttons

Reset buttons can be harmful for users.

Why?

Because one accidental click can erase everything the user typed.

This is especially bad for long forms.

Example:

A user spends 10 minutes filling a job application form.

Then they accidentally click:

```
<button type="reset">Reset form</button>
```

All the form data may disappear.

Simple rule:

Avoid reset buttons unless there is a strong reason to use them.

8. `type="button"`

A button with `type="button"` is a normal button.

Example:

```
<button type="button">Show message</button>
```

Simple meaning:

This button does not submit or reset the form by itself.

In real websites, JavaScript usually gives this button behavior.

Example use cases in real projects:

- show a message
- open a menu
- show/hide a password
- add another form field
- open a modal window

But in this HTML-only curriculum, you only need to understand the HTML meaning.

Beginner rule:

Use `type="button"` when the button should not submit or reset the form.

9. Why Button Type Is Important

A beginner may write:

```
<button>Save</button>
```

This may work, but it is not always clear.

Inside a form, a `button` without a type may behave like a submit button in many cases.

That can cause accidental form submission.

Better:

```
<button type="submit">Save</button>
```

or:

```
<button type="button">Save</button>
```

depending on what you want.

Simple rule:

Always write the button type clearly.

This makes your code easier to understand and safer.

10. `button` vs `input` Buttons

HTML has more than one way to create buttons.

You can use:

```
<button type="submit">Submit</button>
```

or:

```
<input type="submit" value="Submit">
```

Both can create buttons, but they are different.

11. The `button` Element Version

Example:

```
<button type="button">Start Game</button>
```

The `button` element has opening and closing tags.

That means it can contain content.

Example with text:

```
<button type="submit">Create Account</button>
```

Example with text and an icon-like character:

```
<button type="submit">✓ Create Account</button>
```

Simple meaning:

The `button` element is flexible because it can contain content between the tags.

12. The `input` Button Version

The `input` element can also create buttons.

Examples:

```
<input type="button" value="Start Game">  
<input type="submit" value="Submit Form">  
<input type="reset" value="Reset Form">
```

Important:

`input` is a void element.

That means:

- it does not wrap content
- it does not have a closing tag
- it cannot contain button text between tags

Wrong:

```
<input type="button">Start</input>
```

Correct:

```
<input type="button" value="Start">
```

The visible button text comes from the `value` attribute.

Simple rule:

For input buttons, use `value` to set the button text.

13. `button` vs `input` Button Comparison

Feature	<code>button</code>	<code>input type="button"</code>
Has closing tag?	Yes	No
Void element?	No	Yes
Button text location	Between tags	In <code>value</code> attribute
Can contain child content?	Yes	No
More flexible?	Yes	Less flexible

Example `button`:

```
<button type="submit">Submit Form</button>
```

Example `input` button:

```
<input type="submit" value="Submit Form">
```

Beginner recommendation:

Use the `button` element most of the time because it is easier to read and more flexible.

14. Native Browser Validation

Validation means checking whether form data is acceptable.

Native browser validation means the browser checks simple form rules automatically.

You do not need JavaScript for these basic checks.

Examples:

```
<input type="email" required>
```

This can check:

- the field is not empty
- the value looks like an email address

Another example:

```
<input type="text" minlength="3" maxlength="40">
```

This can check:

- the value is not too short
- the value is not too long

Simple meaning:

Native validation is basic validation built into the browser.

15. Client-Side Validation vs Server-Side Validation

Native browser validation is client-side validation.

Client-side means:

The check happens in the user's browser.

This is useful because the user gets quick feedback.

But client-side validation is not enough for security.

Why?

Because users can bypass browser validation.

For example:

- a user can edit HTML in DevTools
- a user can send data directly to the server
- a user can disable some browser behavior

So real websites also need server-side validation.

Server-side means:

The website's server checks the data after it is submitted.

Simple rule:

Client-side validation helps users. Server-side validation protects the application.

For this HTML sprint, you only need to learn the HTML side.

16. The `required` Attribute

The `required` attribute means the field must be filled in.

Example:

```
<input type="text" id="full-name" name="full-name" required>
```

Simple meaning:

The user cannot submit the form while this field is empty.

Full example:

```
<form action="">
  <label for="full-name">Full Name:</label>
  <input type="text" id="full-name" name="full-name" required>

  <button type="submit">Submit</button>
</form>
```

If the user clicks submit without typing a full name, the browser may stop the form and show a message.

17. `required` Is a Boolean Attribute

`required` is a boolean attribute.

A boolean attribute works by being present or absent.

Present means on.

Absent means off.

Correct if the field is required:

```
<input type="email" required>
```

Correct if the field is not required:

```
<input type="email">
```

Wrong idea:

```
<input type="email" required="false">
```

Why is this wrong?

Because `required` is still present.

The browser mainly sees:

```
required
```

So the field is still treated as required.

Simple rule:

Do not write `required="false"` **. Remove** `required` **if the field is not required.**

18. Email Validation with `type="email"`

The email input type tells the browser that the field expects an email address.

Example:

```
<input type="email" id="email" name="email">
```

Simple meaning:

This field expects an email-like value.

Example with label:

```
<label for="email">Email Address:</label>
<input type="email" id="email" name="email">
```

Example with required:

```
<label for="email">Email Address:</label>
<input type="email" id="email" name="email" required>
```

If the user types:

hello

The browser may stop the form because `hello` does not look like an email address.

If the user types:

student@example.com

The browser may accept it as an email-like value.

Important beginner note:

Browser email validation is basic.

It does not prove that the email address really exists.

It only checks whether the value looks like an email format.

Simple rule:

Use `type="email"` for email address fields.

19. The `minlength` Attribute

The `minlength` attribute sets the minimum number of characters.

Example:

```
<input type="text" id="full-name" name="full-name" minlength="3">
```

Simple meaning:

The user must type at least 3 characters.

Example:

```
<label for="username">Username:</label>
<input type="text" id="username" name="username" minlength="4">
```

If the user types:

Sam

That is only 3 characters.

The browser may reject it if `minlength="4"` is required.

Simple rule:

Use `minlength` when the input should not be too short.

20. The `maxlength` Attribute

The `maxlength` attribute sets the maximum number of characters.

Example:

```
<input type="text" id="full-name" name="full-name" maxlength="40">
```

Simple meaning:

The user cannot type more than 40 characters.

Example:

```
<label for="username">Username:</label>
<input type="text" id="username" name="username" maxlength="20">
```

This limits the username to 20 characters.

Simple rule:

Use `maxlength` when the input should not be too long.

21. Using `required`, `minlength`, and `maxlength` Together

You can combine validation attributes.

Example:

```
<label for="full-name">Full Name:</label>
<input
  type="text"
  id="full-name"
  name="full-name"
  required
```

```
    minlength="3"  
    maxlength="40"  
>
```

This means:

- the field cannot be empty
- the value must be at least 3 characters
- the value cannot be more than 40 characters

Another example:

```
<label for="email">Email Address:</label>  
<input  
  type="email"  
  id="email"  
  name="email"  
  required  
  minlength="4"  
  maxlength="64"  
>
```

This means:

- the field cannot be empty
- the value should look like an email address
- the value must be at least 4 characters
- the value cannot be more than 64 characters

Simple rule:

Validation attributes can work together to describe what the input should accept.

22. Form States

Form controls can have different states.

A state describes the current condition of the input.

Sprint 8 focuses on four important states:

- default
- focused
- disabled

- readonly
-

23. Default State

The default state is the normal state of an input.

Example:

```
<input type="email" id="email" name="email">
```

This field is normal.

The user can:

- click it
- focus it
- type in it
- edit it
- submit its value

Simple meaning:

Default state = normal usable input.

24. Focused State

Focused means the input is currently selected.

This can happen when:

- the user clicks inside the input
- the user presses the Tab key and reaches the input
- assistive technology moves focus to the input

Example:

```
<input type="email" id="email" name="email">
```

When the user clicks inside this input, it becomes focused.

Important:

You usually do not write a `focused` attribute in HTML.

Focus is an interaction state.

It happens while the user is using the page.

Simple meaning:

Focused state = the field is currently active or selected.

Why focus matters:

- keyboard users need focus to move through forms
- visible focus helps users know where they are
- screen readers announce focused form controls

Simple rule:

Good forms must work well with keyboard focus.

25. Disabled State

The `disabled` attribute makes a form control unavailable.

Example:

```
<input type="email" id="old-email" name="old-email" disabled>
```

Simple meaning:

The user cannot use this field.

A disabled input:

- cannot be focused
- cannot be edited
- cannot be activated
- is usually skipped by keyboard navigation
- often appears visually faded in the browser

Example:

```
<label for="coupon">Coupon Code:</label>
<input type="text" id="coupon" name="coupon" disabled>
```

This could mean the coupon field is not currently available.

Use disabled when:

- a field is not available yet
- a field depends on another choice
- a feature is not active
- the user is not allowed to use that field

Simple rule:

Disabled = locked and not usable.

26. disabled Is a Boolean Attribute

Like `required`, `disabled` is also a boolean attribute.

Correct disabled field:

```
<input type="text" disabled>
```

Correct normal field:

```
<input type="text">
```

Wrong idea:

```
<input type="text" disabled="false">
```

Why is this wrong?

Because `disabled` is still present.

The field may still be disabled.

Simple rule:

Remove `disabled` if the field should be usable.

27. Readonly State

The `readonly` attribute makes an input readable but not editable.

Example:

```
<input type="text" id="source" name="source" value="Website" readonly>
```

Simple meaning:

The user can see the value, but cannot change it.

A readonly input:

- can usually be focused
- can usually be selected
- cannot be edited
- shows fixed information inside a form

Example:

```
<label for="account-type">Account Type:</label>
<input
  type="text"
  id="account-type"
  name="account-type"
  value="Free Account"
  readonly
>
```

This shows the account type, but the user cannot edit it.

Use readonly when:

- you want to show a fixed value
- the value should be visible in the form
- the user should not change it
- the field is informational but still part of the form display

Examples:

- account type
- reference number
- signup source
- fixed username
- generated order number

Simple rule:

Readonly = visible and focusable, but not editable.

28. `readonly` Is a Boolean Attribute

`readonly` is also a boolean attribute.

Correct readonly field:

```
<input type="text" value="Website" readonly>
```

Correct editable field:

```
<input type="text" value="Website">
```

Wrong idea:

```
<input type="text" value="Website" readonly="false">
```

Why is this wrong?

Because `readonly` is still present.

The browser may still treat the field as readonly.

Simple rule:

Remove `readonly` if the user should edit the field.

29. Disabled vs Readonly

Disabled and readonly are not the same.

This is one of the most important Sprint 8 concepts.

Disabled Example

```
<input type="text" value="Cannot use this" disabled>
```

Simple meaning:

The user cannot use this field.

The user usually cannot focus it or edit it.

Readonly Example

```
<input type="text" value="Can see but not edit" readonly>
```

Simple meaning:

The user can see and focus this field, but cannot edit it.

Comparison Table

Feature	Disabled	Readonly
Can user edit it?	No	No
Can user focus it?	Usually no	Usually yes
Is it usable?	No	Partly yes
Main meaning	Not available	Fixed value
Common use	unavailable field	visible fixed information

Memory rule:

Disabled = locked and not usable.

Readonly = visible and focusable, but not editable.

30. The `value` Attribute

The `value` attribute gives an input a starting value.

Example:

```
<input type="text" value="Website">
```

The input starts with:

Website

This is useful with readonly fields.

Example:

```
<input type="text" id="source" name="source" value="Website" readonly>
```

Simple meaning:

The form shows `Website` as the signup source, and the user cannot change it.

Simple rule:

Use `value` when an input should already contain a starting value.

31. Full Sprint 8 Example Page

This example combines Sprint 2 boilerplate, Sprint 7 form structure, and Sprint 8 buttons, validation, and states.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Job Alert Signup</title>
</head>
<body>
```

```
<h1>Job Alert Signup</h1>

<p>Enter your details to receive job alerts.</p>

<form action="">
  <label for="full-name">Full Name:</label>
  <input
    type="text"
    id="full-name"
    name="full-name"
    required
    minlength="3"
    maxlength="40"
  >

  <label for="email">Email Address:</label>
  <input
    type="email"
    id="email"
    name="email"
    required
    minlength="4"
    maxlength="64"
    placeholder="example@email.com"
  >

  <label for="role">Preferred Role:</label>
  <input
    type="text"
    id="role"
    name="role"
    required
    minlength="2"
    maxlength="50"
  >

  <label for="source">Signup Source:</label>
  <input
    type="text"
    id="source"
    name="source"
    value="Website"
    readonly
  >

  <button type="submit">Create Job Alert</button>
</form>
```

```
</body>  
</html>
```

32. Explanation of the Full Example

Boilerplate

```
<!DOCTYPE html>  
<html lang="en">  
  <head>  
    <meta charset="UTF-8">  
    <meta name="viewport" content="width=device-width, initial-scale=1.0">  
    <title>Job Alert Signup</title>  
  </head>  
  <body>  
    ...  
  </body>  
</html>
```

This creates a complete HTML document.

It includes:

- DOCTYPE
- `<html lang="en">`
- `<head>`
 - charset metadata
 - viewport metadata
 - title
- `<body>`

Main Heading

```
<h1>Job Alert Signup</h1>
```

This gives the page a clear main topic.

Intro Paragraph

```
<p>Enter your details to receive job alerts.</p>
```

This tells the user what the form is for.

Form

```
<form action="">
  ...
</form>
```

This creates the form area.

The empty `action` means the form is not connected to a real server destination yet.

Full Name Field

```
<label for="full-name">Full Name:</label>
<input
  type="text"
  id="full-name"
  name="full-name"
  required
  minlength="3"
  maxlength="40"
>
```

This field collects the user's full name.

Important parts:

- `label` gives the field name
- `for="full-name"` connects to `id="full-name"`
- `type="text"` accepts normal text
- `name="full-name"` names the submitted data
- `required` means it cannot be empty
- `minlength="3"` means at least 3 characters
- `maxlength="40"` means no more than 40 characters

Email Field

```
<label for="email">Email Address:</label>
<input
  type="email"
  id="email"
  name="email"
```

```
    required
    minlength="4"
    maxlength="64"
    placeholder="example@email.com"
  >
```

This field collects the user's email address.

Important parts:

- `type="email"` expects an email-like value
- `required` means it cannot be empty
- `placeholder` gives a short example
- `minlength` and `maxlength` control length

Remember:

The placeholder is only a hint. The label is still required.

Preferred Role Field

```
<label for="role">Preferred Role:</label>
<input
  type="text"
  id="role"
  name="role"
  required
  minlength="2"
  maxlength="50"
>
```

This field collects the user's preferred job role.

Examples:

- Frontend Developer
- Backend Developer
- UI Designer
- QA Tester

Signup Source Field

```
<label for="source">Signup Source:</label>
<input
```

```
type="text"
id="source"
name="source"
value="Website"
readonly
>
```

This field shows a fixed value.

The value is:

Website

The user can see it, but cannot edit it.

This is a good use of `readonly`.

Submit Button

```
<button type="submit">Create Job Alert</button>
```

This submits the form.

The button text clearly explains the action.

33. Common Sprint 8 Mistakes and Fixes

Mistake 1: Writing `required="false"`

Wrong:

```
<input type="email" required="false">
```

Problem:

`required` is a boolean attribute.

If it is present, it is turned on.

Correct if required:

```
<input type="email" required>
```

Correct if not required:

```
<input type="email">
```

Simple fix:

Remove `required` **if the field is not required.**

Mistake 2: Forgetting Button Type

Weak:

```
<button>Save</button>
```

Problem:

The button behavior may not be clear, especially inside a form.

Better:

```
<button type="submit">Save</button>
```

or:

```
<button type="button">Save</button>
```

Simple fix:

Always write the button type.

Mistake 3: Using Reset Button Without Need

Weak:


```
<button type="reset">Reset form</button>
```

This is valid HTML, but it may be bad user experience.

Problem:

The user can accidentally erase all typed data.

Better:

Only use reset if there is a clear reason.

Simple fix:

Avoid reset buttons in normal forms unless truly needed.

Mistake 4: Writing Input Button with Closing Tag

Wrong:

```
<input type="button">Start</input>
```

Problem:

`input` is a void element.

It cannot contain text and should not have a closing tag.

Correct:

```
<input type="button" value="Start">
```

Simple fix:

Use `value` for text on input buttons.

Mistake 5: Using `disabled="false"`

Wrong:

```
<input type="text" disabled="false">
```

Problem:

`disabled` is a boolean attribute.

If it exists, the field is disabled.

Correct disabled field:

```
<input type="text" disabled>
```

Correct usable field:

```
<input type="text">
```

Simple fix:

Remove `disabled` **if the field should be usable.**

Mistake 6: Using `readonly="false"`

Wrong:

```
<input type="text" readonly="false">
```

Problem:

`readonly` is a boolean attribute.

If it exists, the field is readonly.

Correct readonly field:

```
<input type="text" readonly>
```

Correct editable field:

```
<input type="text">
```

Simple fix:

Remove `readonly` **if the field should be editable.**

Mistake 7: Confusing Disabled and Readonly

Weak:

```
<input readonly disabled value="Website">
```

Problem:

This mixes two different states.

If you want the user to see a fixed value, use readonly:

```
<input value="Website" readonly>
```

If the field should not be available at all, use disabled:

```
<input value="Not available" disabled>
```

Simple fix:

Use readonly for fixed visible values. Use disabled for unavailable fields.

Mistake 8: Using Placeholder Instead of Label

Wrong:

```
<input type="email" placeholder="Email Address" required>
```

Problem:

There is no real label.

The placeholder disappears when the user types.

Better:

```
<label for="email">Email Address:</label>
<input type="email" id="email" name="email" placeholder="example@email.com"
required>
```

Simple fix:

Use a label for the field name. Use placeholder only as a hint.

34. Debug This Sprint 8 Code

Broken code:

```
<form action="">
  <label>Email:</label>
  <input type="email" required="false" placeholder="Email Address">

  <input type="button">Submit</input>

  <input type="text" value="Website" readonly="false">
</form>
```

Problems:

1. The label is not connected to the input.
2. `required="false"` is wrong because `required` is boolean.
3. The placeholder is being used like a label.
4. `input` button has wrong text placement and a closing tag.
5. `readonly="false"` is wrong because `readonly` is boolean.
6. The submit control should be a real submit button.

Fixed version:

```
<form action="">
  <label for="email">Email Address:</label>
  <input
    type="email"
    id="email"
    name="email"
```

```
    required
    placeholder="example@email.com"
  >

  <label for="source">Signup Source:</label>
  <input
    type="text"
    id="source"
    name="source"
    value="Website"
    readonly
  >

  <button type="submit">Submit</button>
</form>
```

Why this is better:

- the label is connected with `for` and `id`
- `required` is used correctly
- placeholder is only a hint
- the source field is readonly correctly
- the form has a proper submit button

35. Sprint 8 Mini Project: Job Alert Signup Form

Create a file named:

```
sprint-8-job-alert-signup.html
```

Build a form with:

- full name field
- email field
- preferred role field
- readonly source field with value `Website`
- submit button
- `required` validation
- `minlength`
- `maxlength`

Requirements

Your form should include:

```
<form action="">
```

A full name field:

```
<label for="full-name">Full Name:</label>
<input type="text" id="full-name" name="full-name" required minlength="3"
maxlength="40">
```

An email field:

```
<label for="email">Email Address:</label>
<input type="email" id="email" name="email" required minlength="4"
maxlength="64" placeholder="example@email.com">
```

A preferred role field:

```
<label for="role">Preferred Role:</label>
<input type="text" id="role" name="role" required minlength="2" maxlength="50">
```

A readonly source field:

```
<label for="source">Signup Source:</label>
<input type="text" id="source" name="source" value="Website" readonly>
```

A submit button:

```
<button type="submit">Create Job Alert</button>
```

36. Sprint 8 Practice Questions

Answer these without looking at the notes.

1. What does `type="submit"` do?

2. What does `type="reset"` do?
 3. Why can reset buttons be risky?
 4. What does `type="button"` do by itself?
 5. What is the difference between `button` and `input type="button"`?
 6. Where does button text go in a `button` element?
 7. Where does button text go in an `input` button?
 8. What does `required` mean?
 9. Why is `required="false"` wrong?
 10. What does `type="email"` check?
 11. Does `type="email"` prove that an email address really exists?
 12. What does `minlength` control?
 13. What does `maxlength` control?
 14. What is the default state of an input?
 15. What does focused mean?
 16. Can a disabled input be focused?
 17. Can a readonly input be edited?
 18. What is the difference between disabled and readonly?
 19. When should you use readonly?
 20. When should you use disabled?
-

37. Sprint 8 Self-Check

Before you finish Sprint 8, check your code.

Your form should have:

- a complete HTML boilerplate
 - a clear `h1`
 - a form with `action`
 - every input connected to a label
 - every input with a useful `name`
 - a submit button with `type="submit"`
 - required fields where needed
 - `minlength` and `maxlength` where needed
 - `type="email"` for email addresses
 - a readonly field for fixed visible information
 - no fake boolean values like `required="false"`
 - no closing tag for `input`
 - no placeholder used as the only label
-

38. Sprint 8 Simple Summary

Sprint 8 is about making forms more useful.

The main ideas are:

Buttons

Use buttons to let users submit or control forms.

```
<button type="submit">Submit</button>
```

Button Types

```
<button type="submit">Submit</button>
<button type="reset">Reset</button>
<button type="button">Normal Button</button>
```

Use `submit` to send a form.

Use `reset` carefully because it can erase user input.

Use `button` when the button should not submit or reset by itself.

Native Validation

Use HTML attributes to let the browser check simple rules.

```
<input type="email" required minlength="4" maxlength="64">
```

Boolean Attributes

Boolean attributes work by being present or absent.

Correct:

```
<input required>
```

Incorrect:


```
<input required="false">
```

Form States

Default = normal usable field.

Focused = currently selected field.

Disabled = cannot be used.

Readonly = can be seen and focused, but cannot be changed.

Best Memory Rule

Disabled = locked and not usable.

Readonly = visible and focusable, but not editable.

39. What You Should Be Able to Do After Sprint 8

After completing Sprint 8, you should be able to:

- build a form with a submit button
 - choose the correct button type
 - explain why reset buttons should be used carefully
 - use `button` and `input` buttons correctly
 - add `required` to important fields
 - use `type="email"` for email fields
 - control input length with `minlength` and `maxlength`
 - explain client-side validation in simple words
 - explain why server-side validation is still needed in real projects
 - use `disabled` correctly
 - use `readonly` correctly
 - explain the difference between disabled and readonly
 - fix common beginner form mistakes
-

40. Final Beginner Reminder

Do not try to memorize everything as separate pieces.

Think about the form as a conversation with the user.

A good form says:

- What information do I need? → `label`
- Where should the user type? → `input`
- What kind of data is expected? → `type`
- Is this required? → `required`
- How short or long can it be? → `minlength` and `maxlength`
- Is this value fixed? → `readonly`
- Is this field unavailable? → `disabled`
- How does the user send it? → `button type="submit"`

Simple final rule:

Good HTML forms are clear, usable, and meaningful before any CSS or JavaScript is added.